Proof-of-Inference: End-to-End Emission and On-Chain Enforcement

The activation-trace commitment that makes "no real compute, no mint" a theorem, not a hope

Zoo Labs Foundation
Hanzo AI
Lux Network

Abstract

A companion proof established that the Freivalds matrix-product verifier over $\mathbb{F}_p$, $p = 2^{61}-1$, catches a fabricated product $C \neq A \cdot B$ with probability $\geq 1 - 1/p$ per challenge vector, with *zero* false rejection over an exact integer accumulator. That primitive was correct but *unwired*: nothing fed it operands from a real forward pass, so an adversary could mint by committing to an output it never computed. This paper closes that gap. We define an *activation-trace transcript* — a keccak Merkle commitment over the exact-integer operands of each proof-bearing matmul of a forward pass — and a permissionless *challenger* that opens a beacon-selected matmul and Freivalds-checks it. We prove (Theorem 3) that a prover who returns any output not produced by the committed computation is caught except with probability $\leq (1/p)^k$, and that an honest prover is *never* slashable (Theorem 5). We give a proof-bearing forward (the `ProvableLinear` layer) whose returned value *is* the committed exact-integer result, eliminating the float-shadow unsoundness; an on-chain witness (`ComputeWitnessLib`) and a native precompile (`aivmbridge` selector `0x12000000`) that run the check in the EVM with no off-chain trust; and a slash gate (`OptimisticEvidence`) that pays a bond to a challenger *only* on a verified fraud. Every result is implemented and tested across four languages (Rust prover, Go verifier, Solidity contract, Go precompile) with byte-identical commitments and a pinned cross-language challenge vector; §8 tabulates the suite, including an adversarial red-team battery that exercises each attack the theorems forbid.

1 Setting and the gap this closes

An LLM forward pass is, to ∼95% of its arithmetic, a sequence of matrix multiplications $C^{(\ell)} = A^{(\ell)}B^{(\ell)}$, $\ell = 1, \ldots, L$. Proof-of-Inference binds a coin reward to having *performed* that arithmetic. The verifier is Freivalds' check [1]: for a random r, test $A(Br) \stackrel{?}{=} Cr$ over $\mathbb{F}_p$ in $O(tk + kn + tn)$ — an order cheaper than recomputing C — catching $C \neq AB$ with probability $\geq 1 - 1/p$.

We work over the Mersenne prime $p = 2^{61} - 1$ on the *exact integer accumulator* of the int8 matmul path ($i8 \times i8 \to i64$, whole-K, no cross-block float reduction). Integer arithmetic is bit-identical across CPU/GPU/backends, giving zero false rejection. The companion paper proves the single-product soundness; we take it as Lemma 1 and build the system that feeds it.

Lemma 1 (Single-product soundness, companion)**.** *For integer matrices $A \in \mathbb{Z}^{t\times k}, B \in \mathbb{Z}^{k\times n}, C \in \mathbb{Z}^{t\times n}$ reduced into $\mathbb{F}_p$ entrywise, and k challenge vectors $r_1, \ldots, r_k$ drawn uniformly and independently from $\mathbb{F}_p^n$: if $C = AB$ then $A(Br_j) = Cr_j$ for all j (zero false reject); if $C \neq AB$ then $\Pr[\forall j : A(Br_j) = Cr_j] \leq (1/p)^k$.*

The gap. Lemma 1 is about *operands*. The unsoundness the audit found was that nothing in the deployed system ever produced operands from a real model: the verifier had zero callers, and the mint gate accepted any non-empty "witness". So a prover committed an output hash with no computation behind it and minted. Everything below is the missing wire.

2 The activation-trace transcript

Definition 1 (Matmul leaf)**.** *For a matmul with exact integer operands (A, B, C), its leaf is*

$$\mathsf{leaf}(A, B, C) \triangleq \mathsf{keccak}\big(\, \texttt{DOM} \| \,\mathsf{enc}(A) \,\|\, \mathsf{enc}(B) \,\|\, \mathsf{enc}(C) \,\big),$$

where $\mathsf{enc}(M) = \mathrm{BE}_{32}(\mathit{rows}) \,\|\, \mathrm{BE}_{32}(\mathit{cols}) \,\|\, \big(\mathrm{BE}_{64}(M_{ij})\big)_{ij}$ *is the canonical row-major serialization and* $\texttt{DOM} =$ *''hanzo/poi/matmul-leaf/v1''.*

Definition 2 (Transcript and root)**.** *A forward pass commits its proof-bearing matmuls $(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})_{\ell=1}^{L}$ as the leaf vector $(\mathsf{leaf}_\ell)_\ell$. The* transcript root *$\rho$ is the keccak binary Merkle root over those leaves (odd levels duplicate the last node; the index fold is* $\mathsf{even} \to \mathsf{keccak}(\mathit{node} \| \mathit{sib})$, $\mathsf{odd} \to \mathsf{keccak}(\mathit{sib} \| \mathit{node})$*, identical to the deployed* `_verifyMerkle`*). The prover posts ρ on-chain alongside its claimed output.*

Definition 3 (Opening)**.** *To* open *index ℓ, the prover reveals $(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ and the Merkle path π_ℓ. The opening is* included *iff folding $\mathsf{leaf}(A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ with π_ℓ under the index reproduces ρ.*

Lemma 2 (Binding)**.** *Under collision resistance of* keccak*, an opening at index ℓ that is included under ρ reveals exactly the operands the prover committed at ℓ: producing an included opening with any $(A', B', C') \neq (A^{(\ell)}, B^{(\ell)}, C^{(\ell)})$ requires a keccak collision.*

Proof. Inclusion forces $\mathsf{leaf}(A', B', C')$ to equal the committed leaf at ℓ (the Merkle path and index are fixed by ρ and a second pre-image at any internal node is a collision). Equal leaves with distinct operands is a collision of keccak on the encoded operand string, since enc is injective (fixed-width, length-prefixed). □ □

3 The challenger and end-to-end soundness

Definition 4 (Beacon-bound challenge)**.** *Given a public, post-commitment beacon β (§6), the challenge for index ℓ is $r_{j,i} = \mathrm{BE}_{64}\big(\mathsf{keccak}(\sigma \,\|\, \mathrm{BE}_{32}(j) \,\|\, \mathrm{BE}_{64}(i))[0{:}8]\big) \bmod p$ with seed $\sigma = \mathsf{keccak}(\beta \| \rho \| \mathrm{BE}_{64}(\ell))$, for $j < k$, $i < n$. Modeling* keccak *as a random oracle, the r_j are uniform and independent in $\mathbb{F}_p^n$ and are fixed only* after *ρ is committed.*

Definition 5 (VerifyOpening, ProvesFraud)**.** $\mathsf{VerifyOpening}(\rho, \beta, \mathit{op}) = \mathsf{included}(\mathit{op}, \rho) \wedge \mathsf{Freivalds}_k(A, B, C)$ *with the beacon-bound challenge. The fraud predicate is its dual on the correctness bit:* $\mathsf{ProvesFraud} = \mathsf{included} \wedge \neg\mathsf{Freivalds}_k$.

Theorem 3 (A fabricated matmul is provable)**.** *Suppose the prover's committed transcript contains, at some index ℓ, a matmul with $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$. Then a challenger that opens ℓ produces a witness for which* $\mathsf{ProvesFraud} = \mathsf{true}$ *except with probability $\leq (1/p)^k$ over the beacon. With $k = 2$ and $p = 2^{61} - 1$ this failure probability is $\leq 2^{-122}$.*

Proof. The honest opening of ℓ is included (Lemma 2 gives inclusion for the committed operands), so $\mathsf{included} = \mathsf{true}$. Because $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$, Lemma 1 gives $\Pr[\mathsf{Freivalds}_k = \mathsf{true}] \leq (1/p)^k$, hence $\neg\mathsf{Freivalds}_k$ holds except with that probability, and $\mathsf{ProvesFraud} = \mathsf{true}$. The beacon is drawn after ρ is fixed, so the prover cannot have chosen $C^{(\ell)}$ to satisfy the specific challenge (the pre-fit attack of §6). □ □

Corollary 4 (No-compute ⇒ provable fraud)**.** *A permissionless watcher that re-executes the committed trace off-chain finds every index where* $C^{(\ell)} \neq A^{(\ell)}B^{(\ell)}$ deterministically *(it recomputes* $A^{(\ell)}B^{(\ell)}$ *in the clear) and opens one. Thus any forward pass whose committed output was not actually computed contains at least one such* ℓ *and is slashable except with probability* $\leq (1/p)^k$*.*

Theorem 5 (An honest prover is never slashable)**.** *If every committed matmul satisfies* $C^{(\ell)} = A^{(\ell)}B^{(\ell)}$*, then for every index* ℓ*, every beacon* β*, and every (even adversarially chosen) opening submitted by a griefer,* $\mathsf{ProvesFraud} = \mathsf{false}$*.*

Proof. A griefer's opening either (i) reveals the committed operands, in which case $\mathsf{Freivalds}_k = \mathsf{true}$ by zero-false-reject (Lemma 1), so $\neg\mathsf{Freivalds}_k = \mathsf{false}$; or (ii) reveals different operands, in which case $\mathsf{included} = \mathsf{false}$ except for a keccak collision (Lemma 2). Either way $\mathsf{ProvesFraud} = \mathsf{included} \wedge \neg\mathsf{Freivalds}_k = \mathsf{false}$. □ □

4 Sound emission: the output is the committed value

Theorems 3–5 bind *committed matmuls*. They are only meaningful if the value the model *returns* is determined by those matmuls. The `ProvableLinear` layer makes this hold by construction.

Definition 6 (Proof-bearing linear)**.** *For int8 weights* $W \in [-127, 127]^{o\times m}$ *with public per-row dequant scales* $s^W \in \mathbb{R}^o$*, the layer on input* $x \in \mathbb{R}^{t\times m}$ *computes the per-row symmetric int8 quantization* $(\bar{x}, s^x) = \mathsf{quant}(x)$ *(deterministic:* $s^x_r = \max_j |x_{rj}|/127$*,* $\bar{x}_{rj} = \mathrm{round}(x_{rj}/s^x_r)$*), forms* $C = \bar{x}\, W^\top$ *on the exact* $i64$ *accumulator, commits* $(\bar{x}, W^\top, C)$*, and returns* $y_{ro} = C_{ro}\, s^x_r\, s^W_o$*.*

Proposition 6 (Output binding)**.** *The returned* y *is a fixed, public function* $\mathsf{dequant}(\cdot)$ *of the committed* C *and the committed/public scales. Hence any party can recompute* y *from the opened final-layer* C *and reject a prover that claims a different output; and two honest executions emit byte-identical transcripts.*

Proof. quant and dequant are deterministic given x and the public scales, and C is the exact integer product (no floating-point reduction), so it is reproducible across backends. $y = \mathsf{dequant}(C)$ is then determined by the committed C; equality of two honest runs follows from determinism of every step. □ □

Remark 1 (Why exact integer, not a float shadow)**.** *A kernel that accumulates int8 block dots in* `f32` *(the common fast path) is non-associative and not bit-identical across hardware; committing its* $i64$ *"shadow" would commit a computation different from the returned value, breaking Proposition 6.* `ProvableLinear` *therefore* is *the computation: the returned value is dequantized directly from the committed exact* C*. The throughput cost of the exact accumulator is the price of a reproducible proof and is paid only on proof-bearing inferences.*

5 Full-graph composition: chaining across attention and MoE

Theorem 3 binds each matmul; Proposition 6 binds the output to the final matmul. What remains is to bind the *whole graph*: a prover could commit correct matmuls computed on *unrelated* inputs — a transcript that verifies per-matmul but is not a real forward pass on the committed prompt. We close this with a typed computation trace.

Definition 7 (Computation trace)**.** *A trace is a sequence of typed ops $o_1, \ldots, o_T$ over hash-named activations. Each op commits a leaf* $\mathsf{keccak}(\mathit{DOM}_{\mathrm{op}} \,\|\, \mathsf{kind} \,\|\, (\mathsf{cm}(I))_{I\in\mathrm{in}(o)} \,\|\, \mathsf{params} \,\|\, \mathsf{cm}(\mathrm{out}(o)))$*, where* $\mathsf{cm}(\cdot) = \mathsf{keccak}(\mathit{DOM}_{\mathrm{act}} \| \mathsf{enc}(\cdot))$ *names an activation tensor. Op kinds are* MatMul *(verified by Freivalds), and the deterministic, exact-integer glue* Add *(residual),* ReLU*,* Scale *($1/\sqrt{d}$),* TopKRoute *(MoE router, canonical tie-break),* Gather *(expert dispatch). The trace root is the keccak Merkle root over the op leaves.*

Definition 8 (Well-formed; locally correct)**.** *A trace is* well-formed *if every op's input names are the declared trace input or the output name of an earlier op. An op is* locally correct *if, opened, its revealed tensors match its committed input/output names and it recomputes correctly:* MatMul *by Freivalds under the beacon-bound challenge; a glue op by direct recomputation of its deterministic function.*

Theorem 7 (Composition)**.** *If a trace is well-formed and every op is locally correct, then the committed final activation equals the deterministic forward pass Φ applied to the committed input x_0. Conversely, if the committed output is not $\Phi(x_0)$, then some op is either not well-formed (caught from the leaves) or not locally correct (caught on opening, except with probability $\leq (1/p)^k$ for a* MatMul*).*

Proof. Order the ops topologically (the trace order suffices). Induct on i that op o_i's output tensor equals o_i's true function applied to the true values of its inputs. Base: o_1's inputs are the declared x_0 (well-formedness), so local correctness gives $\mathrm{out}(o_1) = o_1(x_0)$, the true value. Step: o_i's inputs are earlier outputs (well-formedness), which by the inductive hypothesis are their true values; local correctness then gives $\mathrm{out}(o_i)$ the true value. Hence the final output is $\Phi(x_0)$. The converse is the contrapositive: a wrong output forces a wrong op, caught by the corresponding check (Lemma 1 bounds the MatMul case). □ □

Remark 2 (Coverage)**.** *Attention's spine* $\mathrm{ctx} = \mathrm{softmax}(QK^\top/\sqrt{d})\,V$ *is two* MatMul*s and a* Scale *($QK^\top$, scale, then $\cdot V$); an MoE block is* MatMul *(router) →* TopKRoute *→* Gather *→* MatMul *(up) →* ReLU *→* MatMul *(down) →* Add*. Both are well-formed traces in the implementation, every op of which verifies (§8). The softmax normalization is a* Softmax *op realized as a* deterministic fixed-point integer *reduction (Q16, max-shift for stability, an I-BERT-style integer* exp*): pure-integer, hence bit-reproducible across backends, so the challenger recomputes it and checks the committed output exactly — the attention block is the complete spine $QK^\top \to$ scale $\to$ softmax $\to \cdot V$, with no floating-point reduction anywhere in the committed trace.*

6 The post-commitment beacon

Soundness (Theorem 3) requires the challenge to be unpredictable when the prover chooses C. If the seed depended only on ρ (which the prover knows), a prover could fabricate $C' = AB + \Delta$ with $\Delta r = 0$ for the resulting r and pass. We therefore bind the beacon to a value fixed *after* the prover's commitment is mined: $\beta = \mathsf{keccak}(\mathsf{blockhash}(b_0) \,\|\, \mathsf{id})$ where b_0 is the commitment's block. A transaction cannot contain its own block's hash, so a prover choosing C before signing cannot predict β; blockhash is available for 256 blocks, bounding the challenge window. The pre-fit attack is exactly attack 6 of the red-team battery (§8): a forgery tuned to a guessed r_1 is shown to pass r_1 and fail a fresh r_2.

7 On-chain enforcement, with no off-chain trust

The witness is checked in the EVM, not asserted by a watcher. Two implementations share one wire format and one field:

- **Reference (Solidity).** `ComputeWitnessLib.verifyOpening` runs Freivalds over $\mathbb{F}_p$ via `mulmod`/`addmod` plus the Merkle fold and the beacon challenge, byte-identical to the prover. Cost is $O(tk+kn+tn)$ field ops — affordable for a bounded slice; a full layer is challenged in slices.
- **Native (precompile).** `aivmbridge` at `0x0300...0004`, selector `0x12000000`, decodes the opening wire and calls the canonical Go verifier, returning $(\mathsf{included} \mid \mathsf{freivaldsOK} \ll 1)$. Gas is base + (field ops) $\cdot\, c$, priced so a 256-wide slice costs ∼1M gas and a 4096-wide layer is unaffordable, forcing bounded slices; `RequiredGas` equals the metered consumption.

Proposition 8 (Gate soundness)**.** *The slash gate* `OptimisticEvidence.challenge` *transitions a commitment to* FRAUDULENT *and pays the bond to the caller* iff ProvesFraud *holds for an opening under the committed root and the commit-block beacon. Consequently (Theorems 3, 5) a fabricated forward pass is slashable except with probability* $\leq (1/p)^k$*, and an honest prover is never slashable.*

Proof. By inspection the gate calls ProvesFraud with $\beta =$ blockhash(commitBlock) and slashes exactly on its truth, within the open window (after which the commitment finalizes). The two theorems then transfer. □ □

8 Implementation and adversarial evidence

Every definition and theorem above is realized in code, with the commitment, the field, the Merkle fold, and the challenge derivation *byte-identical* across the prover, the watcher, the contract, and the precompile (enforced by a pinned cross-language golden vector that breaks one suite if either side drifts). The honest path is exercised on a real SwiGLU FFN forward; the adversarial path on a battery that instantiates each attack the theorems forbid.

Component	What it realizes	Tests
`hanzo-engine/poi.rs`	exact matmul; keccak; Go-parity challenge derivation; Freivalds	
`hanzo-engine/poi_transcript.rs`	leaf, Merkle root/proof, `verify_opening` (§2–3)	
`hanzo-engine/poi_forward.rs`	`ProvableLinear`, `prove` (§4)	18/18
`hanzo-engine/poi_graph.rs`	`GraphTrace`, typed ops incl. fixed-point softmax, chaining (§5)	8/8
`crypto/promptseal` (Go)	post-quantum (X-Wing) prompt seal (§9)	6/6
`crypto/attestation` (Go)	TEE quote verifier: ECDSA + x509 chain + binding (Prop. 10)	6/6
`crypto/poi` (Go)	transcript + `VerifyOpening` + wire codec + adversarial + scaling (§10)	pass
`ComputeWitnessLib.sol`	on-chain Freivalds + Merkle + beacon (§7)	5/5
`OptimisticEvidence.sol`	the slash gate, `ProvesFraud`-driven (Prop. 8)	18/18
`aivmbridge` precompile	native verification, selector `0x12`	5/5
`tests/poi_adversarial.rs`	red-team battery (below)	pass

Red-team battery (each maps to a theorem).

1. Fabricate the final-layer output ⇒ caught (Thm. 3).
2. Fabricate a hidden intermediate matmul ⇒ caught when opened; honest layers still verify.
3. Sparse deep cheat (one wrong entry in a 256-deep matmul) ⇒ caught.
4. Swap the output at open time ⇒ Merkle inclusion fails (Lemma 2).
5. Tamper an input at open time ⇒ inclusion fails (the leaf binds A).
6. Pre-fit C to a *guessed* challenge r_1 ⇒ passes r_1, fails the beacon-fresh r_2 (§6).
7. Griefer submits a fabricated opening not under the honest root ⇒ cannot slash (Thm. 5).
8. Quantization lie (claim $y \neq \mathsf{dequant}(C)$) ⇒ detectable by public recompute (Prop. 6).
9. Determinism: two honest machines emit the byte-identical root (zero false reject).
10. Honest forward verifies under *every* beacon.

9 Confidentiality: the operator never sees plaintext

Soundness binds *correctness*; it says nothing about *who sees the prompt.* The default path handed the winning operator the plaintext prompt by hash. We close this (audit G9) with a single-shot sealing envelope: each operator publishes a KEM public key in its registry entry, and the requester seals the prompt to it.

Definition 9 (Prompt seal)**.** *With operator key pair* $(\mathsf{pk}, \mathsf{sk})$*, the requester computes* $\mathsf{seal}(\mathsf{pk}, m, \mathrm{aad}) = \mathsf{enc} \,\|\, \mathsf{HPKE.Seal}(\mathsf{pk}, \mathrm{info}, \mathrm{aad}, m)$*, RFC 9180 base mode with* ***X-Wing*** *(the hybrid X25519 + ML-KEM-768 KEM), HKDF-SHA256, and ChaCha20-Poly1305, the associated data* aad *bound to the request (the* $\mathsf{intentID}$*). Only* sk*, inside the operator's compute boundary, opens it. The hybrid KEM makes the seal* post-quantum*: confidentiality holds if* either *X25519 or ML-KEM-768 is unbroken, so a future quantum adversary that breaks X25519 still cannot recover* m*.*

Remark 3 (The whole scheme is post-quantum)**.** *The proof layer needs no upgrade: Freivalds soundness (Lemma 1) is* information-theoretic *(it holds against an unbounded, hence quantum, adversary), and the commitments are keccak (PQ-secure up to Grover, i.e.* 128*-bit PQ at keccak-256). The staking layer is ML-DSA. With the X-Wing seal, every layer of the system is post-quantum.*

Proposition 9 (Confidentiality)**.** *Under the IND-CCA2 security of the HPKE suite: (i) a passive observer, and any operator other than the intended recipient, learns nothing about* m *beyond its length; (ii) two seals of the same prompt are computationally indistinguishable (fresh ephemeral key per seal); (iii) any modification of the sealed blob, or use under a different* aad*, causes* open *to fail (AEAD authentication), so a sealed prompt cannot be replayed under another request.*

Proof. Each property is the corresponding HPKE guarantee in base mode: (i) and (ii) are KEM/DEM IND-CCA2 secrecy with a fresh encapsulation; (iii) is AEAD integrity over $(\mathrm{aad}, \mathrm{ciphertext})$. The wrapper adds only the length-prefixed encapsulated key, which is part of the HPKE ciphertext. □ □

This makes delegated operation non-custodial *on the wire*: the prompt is ciphertext to everyone but the chosen operator. Custody *inside* the operator's boundary is closed by binding the confidentiality key to an attested enclave.

Definition 10 (Attestation quote)**.** *A quote is* $(\mathsf{measure}, \mathsf{rd}, \mathsf{cert}, \mathsf{chain}, \sigma)$*: an enclave measurement* measure*, report data* rd*, an attestation-key certificate* cert *with chain* chain*, and an ECDSA-P256 signature* σ *over* $(\mathsf{measure} \,\|\, \mathsf{rd})$*. It* verifies for key pk *under policy* $(\mathcal{R}, \mathcal{M})$ *iff (1)* cert *chains through* chain *to a pinned root in* $\mathcal{R}$*; (2)* σ *verifies under* cert*'s key; (3)* $\mathsf{measure} \in \mathcal{M}$*; and (4)* $\mathsf{rd} = \mathrm{H}(\mathsf{pk})$*.*

Proposition 10 (In-enclave custody)**.** *If a quote verifies for the operator's confidentiality key* pk *under a policy whose roots are the genuine vendor CA and whose allow-list is the audited enclave image, then the holder of the corresponding* sk *is that enclave image running on genuine hardware. Composed with Proposition 9: a prompt sealed to* pk *is decryptable* only *inside the attested enclave — custody end to end, no plaintext outside verified code.*

Proof. (1) forces the attestation key to be vendor-certified (hardware genuineness); (2) binds the report to that key; (3) forces the audited image; (4) binds pk into the signed report, so the key was generated and held inside that enclave. An adversary lacking the vendor key fails (1) (its own well-formed quote is rejected); altering the image after signing fails (2); a different key fails (4). By Prop. 9 only sk opens the seal, and sk lives only in the enclave. □ □

10 Scaling: verification is per-node-independent

A node verifies an opening with *no* coordination — VerifyOpening is a pure function of the opening, root, and beacon. So the network is embarrassingly parallel: N nodes contribute $N\times$ aggregate verification capacity at constant per-proof latency, and security *strengthens* with N because a fabricated matmul is caught if *any* honest node opens it. Measured single-node latency on a $64 \times 128 \times 64$ challenged slice is ≈ 1.8 ms (≈ 540 verifications/s/node); the per-node-independent projection is:

nodes	aggregate verif./s	per-proof latency	Pr[fraud caught | 10% honest]
1	5.4×10^2	1.8 ms	0.100
10	5.4×10^3	1.8 ms	0.651
100	5.4×10^4	1.8 ms	0.9999
1000	5.4×10^5	1.8 ms	1.000

The last column is $1 - (1 - h)^N$ for an honest fraction $h = 0.1$ of N nodes independently spot-checking: detection is near-certain by $N = 100$. Throughput is linear because nodes are separate machines with no shared state; the cost the network pays per challenged matmul is one Freivalds check, which is why a single bounded slice suffices regardless of model size.

11 Scope: what is proven and what is assumed

- **Proven (here, on code).** Per-matmul soundness end to end (Thm. 3); no-false-slash (Thm. 5); output binding for the proof-bearing linear (Prop. 6); *full-graph composition* across attention and MoE including the fixed-point softmax (Thm. 7, §5); *post-quantum prompt confidentiality* (Prop. 9); *in-enclave custody* via attestation (Prop. 10); on-chain enforcement with native and reference verifiers (Prop. 8); per-node-independent linear scaling (§10); cross-language byte parity.
- **Assumed.** Collision/pre-image resistance and random-oracle behavior of keccak; the field bound of Lemma 1 (companion); IND-CCA2 of the (post-quantum, X-Wing) HPKE suite

(Prop. 9); unforgeability of ECDSA-P256 and a genuine pinned vendor CA for attestation (Prop. 10); an honest majority of block producers for the blockhash beacon over the challenge window (a stronger VRF beacon strengthens this without changing the theorems).

- **Out of scope (named, not hidden).** Platform-specific quote *parsing* (the SGX/TDX DCAP and SEV-SNP byte layouts map into the canonical quote of §9; this paper owns the verification soundness, not each vendor's wire format).

References

[1] R. Freivalds, *Probabilistic machines can use less running time*, IFIP Congress, 1977.